EchoPin code reference: answers for code questions

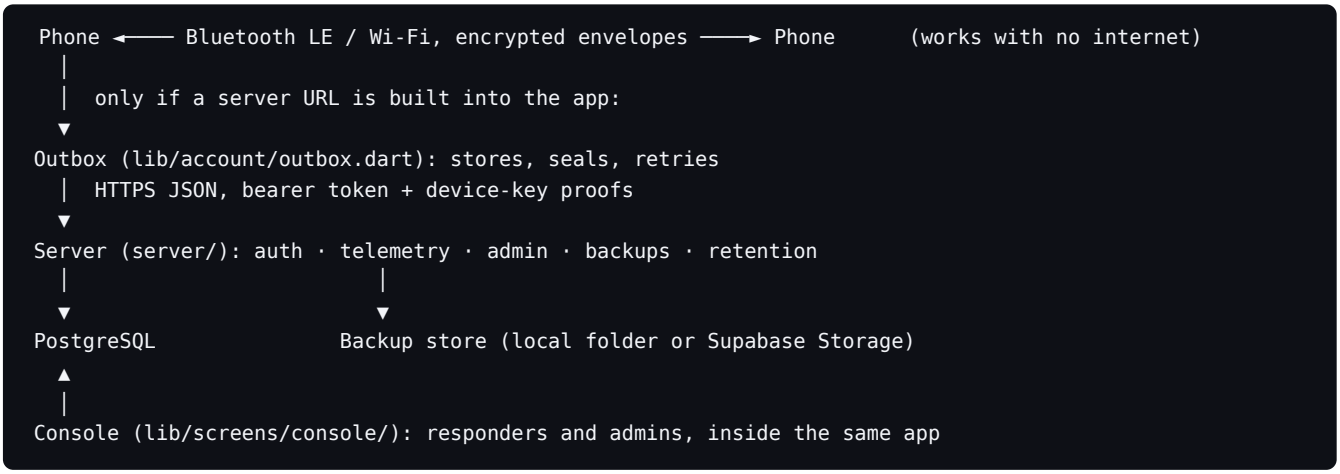
A quick-lookup book for questions after the talk. Every claim points to the function that does it, as `file:line`. Line numbers are as of 8 November 2026 on branch `update`. If they've drifted, search for the function name.

How to use it under pressure. 1. Find the topic in the contents, or jump to the rapid-fire answers. 2. Say the one-line answer. 3. If asked to prove it, open the `file:line` and read the comment above the function; most security-relevant code has a `SECURITY:` note explaining why.

Contents

- The whole system in 60 seconds
- A. Phone-to-phone protocol
- B. Accounts on the phone
- C. Store and forward: outbox and recorder
- D. Sealing records: one lock or two
- E. Server sign-in and accounts
- F. Console and break-glass
- G. Audit log
- H. Keys
- I. Backups and retention
- J. HTTP layer
- K. Database tables
- L. Configuration switches
- M. Tests: what proves what
- Rapid-fire answers
- Numbers to remember

The whole system in 60 seconds



Layer	Where	Language
App	`lib/`	Flutter / Dart
Shared wire code	`packages/echopin_protocol/`	Dart (used by app *and* server)
Server	`server/`	Dart (`shelf`, `postgres`)
Guided tour of every file	`docs/CODE_MAP.md`	

Layer	Where	Language
Protocol spec	`docs/PROTOCOL.md`	
Security review	`docs/SECURITY_REVIEW.md`	

A. Phone-to-phone protocol

What	Where	Key facts
Invite codes	`lib/services/session_codec.dart:41` `SessionCodec`	`ECHO-XXXX-XXXX-XXXX-XXXX-CC`; 16-character body from a 30-letter alphabet with no look-alikes (no I, L, O, U, 0, 1); 2-character checksum (`checksum`, line 157) catches typos
Everything derived from the code	`lib/security/session_credentials.dart:43` `SessionCredentials.derive`	HKDF-SHA256 with salt `echopin/v2/hkdf-salt` produces three values: the **public session id** (8 chars, advertised), the **message key** (32 bytes), and the **server tag** (16 bytes; see F)
Per-session identity	`lib/security/session_identity.dart:59` `SessionIdentity.generate`	Fresh Ed25519 key pair per session. Peer id = 8 chars of SHA-256(public key), so devices can't be linked across sessions
Safety codes	`lib/security/session_identity.dart:30` `safetyCodeFor`	12 digits from SHA-256(`echopin/v2/safety:<session>:` + public key); compared in person
Seal a message	`lib/security/secure_messenger.dart:78` `SecureMessenger.seal`	Inner bytes = kind, sender, recipients, counter, time, body; signed with Ed25519, then XChaCha20-Poly1305 with a random 24-byte nonce
Open a message	`lib/security/secure_messenger.dart:131` `SecureMessenger.open`	Rejects: bad decryption, bad signature, **key change for a known peer** (pinned on first use), clock skew > ±2 min, replays, floods, full session
Replays	`lib/security/replay_guard.dart:25` `ReplayGuard.check`	Per-sender 64-message sliding window over the counter
Flooding	`lib/security/rate_limiter.dart:5` `RateLimiter`	Token bucket per sender: 5 tokens, 1 back every 500 ms
Session logic	`lib/session/session_engine.dart:96` `SessionEngine`	hello / ping / receipt / reply / bye; hello every 20 s; retries every 4 s; check-in cooldown 10 s; 20-member cap; `ping` line 280, `respond` line 364, `_onFrame` line 389
Limits	`lib/security/wire.dart`	Body ≤ 1 KiB, ≤ 32 recipients, names ≤ 24, notes ≤ 80 characters
Distance noise	`lib/services/location_engine.dart:61` `noiseOffsetFor`, `:74` `obfuscatePeer`	Bounded jitter, fixed per person per minute, applied before anything is drawn. **Distance comes from Bluetooth signal strength, never GPS.**

If asked "can someone fake being a friend?" Not without the invite code. Messages need the session key to decrypt and a signature from the sender's key. After the first message, a peer's key is pinned, and a different key for the same id raises a security warning. Safety codes let people verify in person.

B. Accounts on the phone

All in `lib/account/`. Off unless built with `--dart-define=ECHOPIN_SERVER_URL=…`.

Function	Where	What it does
`serverUrlAllowed`	`account_provider.dart:577`	Release builds only talk HTTPS (loopback excepted); a plain-`http` release build behaves as if no server is set

Function	Where	What it does
`DeviceKeyStore.keyPair`	`device_key.dart:21`	This install's Ed25519 device key in the OS keystore (falls back to app storage where there is none)
`AccountProvider.signIn` / `_signIn`	`account_provider.dart:196` / `:218`	Signs `echopin/v1/device-proof|login|<email>|<time>` with the device key; retries once on `device_proof_invalid` (clock fix); makes a fresh key if this device was removed
`_serverNow`	`account_provider.dart:94`	Device time + offset learned from the server's `Date` header (`api_client.dart:84`), so a wrong phone clock doesn't break sign-in
`authed`	`account_provider.dart:291`	Every signed-in request; on 401 refreshes once and retries
`_refresh` / `_doRefresh`	`account_provider.dart:313` / `:321`	**One refresh at a time** (two at once would look like token theft). Signs the refresh with the device key. A refused *proof* keeps the session; a revoked or expired session clears it
`verifyEmail`	`account_provider.dart:158`	Sends code **and** password (anti pre-hijack, see E)
`ensureEscrow` + `_pinCustodian`	`account_provider.dart:370` / `:396`	Caches the server's sealing keys; never drops a custodian key once seen; a key built into the app always wins
`importCustodianKey` / `openInnerSeal`	`account_provider.dart:420` / `:456`	Responder side of the second seal (see D)
`EchoPinApi.send`	`api_client.dart:56`	75 s timeout (free hosts sleep); every failure becomes `OfflineError` or `AccountError(code, message)`

If asked "where are tokens stored?" In the same key-value store as the device key: the OS keystore where available (`secure_kv_store.dart`). Only token *hashes* are on the server.

C. Store and forward: outbox and recorder

Function	Where	What it does
`Outbox.enqueue`	`outbox.dart:180`	Saves an upload on the device first; session records are **sealed before they're stored**
`Outbox._sealIfPossible`	`outbox.dart:286`	Inner custodian seal (if configured), then outer escrow seal; contexts bind each layer to the record's id
`Outbox.flush` / `_flushOnce`	`outbox.dart:203` / `:229`	Sends when online, with backoff; client-made UUIDs make retries idempotent; items belong to the account that made them
`newUuid`	`outbox.dart:353`	Random version-4 UUIDs
`SessionRecorder._onChange` / `_diff`	`session_recorder.dart:86` / `:130`	Turns session changes into events (joins, SOS, check-ins, crossings). No coordinates ever
`_checkpoint` / `_finish`	`session_recorder.dart:330` / `:337`	Upload within seconds of an SOS or "need help", every 10 minutes, and a final record at the end
`_scheduleDraft` / `_recoverDraft`	`session_recorder.dart:417` / `:434`	Survives the app being killed: the draft is uploaded on next start
`_Log.body`	`session_recorder.dart:510`	What's sent: counts in the clear (for the index), the record itself sealed, plus the **session tag**

D. Sealing records: one lock or two

Function	Where	Key facts
`SealedBox.seal`	`packages/echopin_protocol/lib/src/sealed_box.dart:57`	Fresh X25519 key pair per record → shared secret with the recipient → HKDF-SHA256 → XChaCha20-Poly1305. Associated data = `echopin/v1/seal|<key id>|<context>`
`SealedBox.open`	`sealed_box.dart:89`	Needs the recipient's private key **and** the same context; a blob moved to another record fails
`SealedBox.checkStructure`	`sealed_box.dart:125`	What the server checks on upload: shape only, it can't read inside
`SealedBox.keyIdFor`	`sealed_box.dart:49`	Short id of a public key; the app refuses a key whose id doesn't match its bytes (`account_models.dart:109`)
`CustodianKeys`	`server/lib/src/security/custodian.dart:23`	Accepted custodian public keys (newest first, old ones kept for queued uploads)
Responder opens the inner lock	`account_provider.dart:456` `openInnerSeal`, called from `lib/screens/console/console_sessions.dart` `_unseal`	Private custodian key stays in the device's secure storage; never sent anywhere
Create a custodian key	`server/bin/echopin_admin.dart` `init-custodian`	Prints the public half (for the server) and `ECHOPIN-CUSTODIAN-1:…` (for responders)

Two layers:

```
record ──seal(custodian public key, "session-log-custodian:<id>")──► inner
inner  ──seal(escrow public key,    "session-log:<id>")────────────► outer   (uploaded)
```

If asked "so who can read a record?" Only someone who gets through the audited break-glass on the server *and* holds the custodian key on their device. The server alone can't. A stolen database plus the server's keys can't.

E. Server sign-in and accounts

All in `server/lib/src/auth/auth_service.dart` unless noted.

Function	Line	What it does / why
`register`	187	Same 202 answer and ≥ 600 ms for new and taken emails; **always hashes** (so timing can't tell); taken email → owner gets a heads-up email instead; checks breached passwords
`verifyEmail`	280	Code first (burns an attempt), then the **sign-up password**, then a compare-and-set activation. Stops "pre-hijacking": someone re-registering your pending email can't get *their* password activated by *your* code
`login`	345	Order: per-IP limit → **device proof** → lockout (skipped for the account's own registered devices) → password (dummy hash for unknown emails) → status → two-step code → tokens

Function	Line	What it does / why
`_recordFailure`	967	Counts failures per **email** (even with no account) under a row lock; lock 15 min, doubling to 24 h; forgotten after a day
`_lockedUntil`	950	Reads the lock from `login_throttle`
`refresh`	491	Rotates the refresh token; **reuse of an old one revokes the whole family** and is audited; needs a fresh device proof over a digest of the token (`refreshSubject`, 597)
`authenticate`	600	Every request: token hash → session → device → account, all re-checked live (revocation and role changes are instant); also reads whether two-step is on
`AuthContext.require`	58	Role check used by every console method; **staff without two-step get `two_step_required`**
`_createSession`	1069	Opaque random tokens; only SHA-256 hashes stored; access 30 min, refresh 30 days (**7 days** for devices without a keystore)
`forgotPassword` / `resetPassword`	655 / 666	Uniform answers; reset signs out every device and clears the lock
`changePassword`	699	Needs the current password; signs out other devices
`totpSetup` / `totpEnable` / `totpDisable`	728 / 760 / 800	Enable **needs the password** (a stolen 30-minute session can't attach someone else's authenticator); secret encrypted with the data key
`_issueCode`	1145	8-digit codes, 15 minutes, **at most 10 per account per day** (counted in the database)
`_consumeCode`	1200	5 guesses per code; every failure gives the **same** message (no "tries left")
`_uniformTime`	920	Pads public endpoints to ≥ 600 ms
`_mail`	1239	Sends in the background; failures logged and audited as `mail.failed`
`_rejectBreached`	931	Uses `HibpBreachChecker` (`breach_check.dart:37`)
`normalizeEmail`	1306	Lower-cases; refuses commas, brackets, quotes, control characters

Supporting pieces:

What	Where	Key facts
Password hashing	`server/lib/src/auth/passwords.dart:30` `hash`, `:45` `verify`	Argon2id, 19 MiB, 2 passes (OWASP minimum), on a separate isolate; constant-time compare (`constantTimeEquals`, line 99)
Password rules	`passwords.dart:110` `passwordProblem`	10–256 characters, not too repetitive, no common words unless 16+, not containing your email name
Breached passwords	`server/lib/src/auth/breach_check.dart:45`	Have I Been Pwned range API: sends the first 5 hex characters of SHA-1 only, with padding; **fails open** if unreachable (logged)
Two-step codes	`server/lib/src/auth/totp.dart:37` `Totp.verify`	RFC 6238, 30 s steps, ±1 step drift, replay refused via last-used step
Tokens	`server/lib/src/security/tokens.dart:10`	32 random bytes from `Random.secure()`; `emailCode` line 25; codes stored as SHA-256(account | purpose | code)
Rate limits	`server/lib/src/security/rate_limit.dart:24` `DbRateLimiter`	One atomic upsert in PostgreSQL per check; survives restarts; shared by every instance
Device proofs	`packages/echopin_protocol/lib/src/device_proof.dart:42` `DeviceProof.verify`	Ed25519 over `echopin/v1/device-proof|purpose|subject|time`; ±5 min

F. Console and break-glass

All in `server/lib/src/admin_service.dart`. **Every method checks the role itself** (`ctx.require(...)`), so a routing mistake can't expose anything.

Function	Line	Who	What it does
`overview`	60	responder+	Tiles and 14-day charts; SOS counted per **session**, not per upload
`accounts` / `account`	168 / 203	admin	People list and detail; lock state comes from `login_throttle`
`updateAccount`	250	admin	Role, status, `unlock`, `resetTwoStep`; can't change yourself; can't remove the last admin; disabling or resetting two-step signs them out everywhere
`reports` / `report` / `updateReport` / `addReportNote`	416 / 480 / 530 / 583	responder+	Triage; **reporter emails only for admins**; closing needs a resolution
`sessions`	608	responder+	One row per session from each phone's newest upload; shared events use **max**, not sum; flags `unproven`; **person searches are audited** (`sessions.searched`)
`session`	694	responder+	Every upload + who opened what; **audited** (`session.viewed`); **session tag check**: a tag counts as proven only if a strict majority of *people* share it; ties show as unknown
`unseal`	788	responder+	Reason ≥ 10 characters → 30 per hour limit → **grant and audit row committed before decryption** → peel the server layer → return content, or the inner seal if double-sealed
`_alertAdmins`	883	—	Emails every active admin: who, which session, reason, case number
`auditEntries` / `verifyAudit`	912 / 948	admin	Audit list; chain check including the backup anchor

Uploads arrive through `server/lib/src/telemetry_service.dart`: `addSessionLog` (line 66) is idempotent on the client's UUID, checks the escrow key id, the custodian key id, `CUSTODIAN_REQUIRED`, and the 16-byte session tag. `addReport` (137) never stores a safety report below *high*.

G. Audit log

`server/lib/src/security/audit.dart`

Function	Line	Key facts
`recordIn`	79	Writes inside the caller's transaction (the action and its audit row commit together); an advisory lock keeps one chain tip
`_hash`	192	`HMAC-SHA256(audit key, previous hash ‖ canonical row)`
`verify`	125	Checks the backup anchor first (catches rows cut off the end), then walks every row
`canonicalJson`	232	Sorted keys, so equal details hash equally

Three layers: database triggers refuse UPDATE / DELETE / TRUNCATE (migration 3); each hash covers the previous row; the HMAC key lives in `ECHOPIN_KEYS`, never in the database. **Anchor:** each backup stores the newest entry's id and hash next to the backups (`backup_service.dart:291` `anchorName`, `:299` `latestAnchor`) and prints it to the server log.

H. Keys

`server/lib/src/security/keys.dart`

Key	Purpose
Escrow (X25519, can rotate, old ones kept)	Outer seal on session records
Data (AES-256)	Encrypts two-step secrets (`encryptData` 215 / `decryptData` 224)
Backup (AES-256)	Encrypts backups
Audit MAC (derived from data key, line 56)	Signs the audit chain
Custodian (X25519, **not on the server**)	Inner seal; public half in `CUSTODIAN_PUBLIC_KEYS`

`loadServerKeys` (301): from `ECHOPIN_KEYS` (hosted) or `KEYS_DIR` (self-hosted). **Refuses to invent keys** for a database that already has data, because new keys would make every sealed record unreadable. Bundles: `newBundle` 129, `rotateBundle` 137, `fromBundle` 162.

I. Backups and retention

`server/lib/src/backup/backup_service.dart`

Function	Line	Key facts
`run`	145	`pg_dump` of EchoPin's tables only → encrypt → store → record → anchor → prune; one at a time (unique index, migration 5)
`encryptStream` / `decryptStream`	439 / 483	Format EPBK1: AES-256-GCM in 1 MiB frames; each frame's associated data = magic + index + "final" flag, so reordering or truncation fails
`verifyFile` / `restoreFile`	345 / 363	Command line only, never a console button; restore leaves Supabase's own schemas alone (`restoreList` , 405)
`startSchedule`	88	Daily at `BACKUP_HOUR_UTC` , keeps `BACKUP_RETENTION` copies

`server/lib/src/retention_service.dart:61` `RetentionService.run`: daily deletion of session records (365 days), diagnostics (90), closed reports (730), expired codes, sessions and counters. One `retention.purged` audit entry per run. **The audit log is never deleted.**

J. HTTP layer

`server/lib/src/api/http.dart` and `server/lib/src/api/routes.dart` (every `/v1` endpoint).

Function	Line	Key facts
`clientIp` + `ClientIpPolicy`	252 / 226	Real client address from an edge-overwritten header (`CLIENT_IP_HEADER` , Render: `true-client-ip`) or counted from the **right** of `X-Forwarded-For` ; never the leftmost entry
`readJson`	65	64 KiB body limit, enforced while streaming
`errorsAndLogging`	156	Errors as `{"error": code, "message": sentence}` ; logs one line per request, never bodies or tokens
`securityHeaders`	183	`nosniff` , `DENY` framing, `no-store` , HSTS
`cors`	198	Only listed origins; exposes `Date` so the web app can learn the clock

---

K. Database tables

`server/lib/src/db/migrations.dart` , applied in order under an advisory lock.

Migration	Tables
1 (line 17)	`accounts`, `email_codes`, `devices`, `auth_sessions`
2 (86)	`diagnostics`, `session_logs`, `unseal_grants`, `reports`, `report_notes`
3 (165)	`audit_log` + immutability triggers, `backups`
4 (207)	checkpoint columns on `session_logs`
5 (215)	one running backup at a time
6 (220)	`rate_hits`, `login_throttle`, custodian key id + session tag on `session_logs`, `secure_storage` on `devices`

Every query uses named parameters (`@name`), never string-built SQL from user input; search text has `LIKE` wildcards escaped (`admin_service.dart` `_escapeLike`).

L. Configuration switches

`server/lib/src/config.dart`. Full table in `docs/SERVER.md`.

Variable	Default	Effect
`ECHOPIN_KEYS`	(required when hosted)	All server keys as one secret
`CUSTODIAN_PUBLIC_KEYS`	unset	Turns on double sealing
`CUSTODIAN_REQUIRED`	on if keys set	Refuse single-sealed uploads
`REQUIRE_STAFF_TWO_STEP`	on	Staff need two-step for the console
`PASSWORD_BREACH_CHECK`	on	HIBP check
`UNSEAL_ALERTS`	on	Email admins on every opening
`RETAIN_*_DAYS`	365 / 90 / 730	Automatic deletion
`TRUST_PROXY`, `CLIENT_IP_HEADER`, `TRUSTED_PROXY_HOPS`	off / unset / 1	Where the client IP comes from
`ECHOPIN_DEV`	off	Refused unless the database is local
`CORS_ORIGINS`	none	`*` refused outside development

M. Tests: what proves what

Suite	Count	Run with	Covers
App	213	`flutter test`	Protocol, engine, providers, accounts, outbox, recorder, widgets
Server	62	`cd server && bash tool/test.sh`	Every endpoint against a throwaway PostgreSQL; `security regressions` and `hardening` groups
Protocol	6	`cd packages/echopin_protocol && dart test`	Sealing and device proofs
Live, end to end	30 checks + UI run	`integration_test/live_server_test.dart`	Real compiled server and database; the real app clicked through on a virtual display

Key regression tests (all in `server/test/api_test.dart`): pre-hijack, parallel lockout, code cap, spoofed IP, keyed audit chain, two-step reset, double seal, session-tag ties, SOS counting, retention, audit anchor.

Rapid-fire answers

Why not just use the IMEI? Other apps can read it, anyone can copy it, modern phones mostly hide it, and it can't be revoked. A device key's private half never leaves the phone, and every refresh is signed with it. → `device_key.dart:21`, `device_proof.dart:42`

Do you store locations? No. There's no latitude or longitude anywhere. Distance is Bluetooth signal strength on your own phone, with noise added before display. Records contain events, never coordinates. → `location_engine.dart:74`, `session_recorder.dart:510`

Can the server read emergency records? With a custodian key configured, no: it can only remove its own outer layer. Without one, only through the audited break-glass path. → `admin_service.dart:788`, `outbox.dart:286`

What stops a responder snooping? A written reason; an audit row committed before decryption; every admin emailed; 30 openings per hour; two-step required for staff; person searches audited. → `admin_service.dart:788`, `:883`, `auth_service.dart:58`

What if someone steals the database? Tokens are stored as hashes, passwords as Argon2id, two-step secrets encrypted with a key that isn't in the database, records sealed. With a custodian key they stay sealed even with the server keys.

What if someone steals a refresh token? Useless without the device key that must sign each refresh. If the genuine device then refreshes, the reuse is detected, every token in that family is revoked, and it's audited. → `auth_service.dart:491`

Can someone lock me out by guessing my password? Only from devices you've never used. Your own registered phones skip the lock, and a password reset clears it. → `auth_service.dart:345`, `:967`

Can someone find out if an email has an account? No: same answers, same timing (≥ 600 ms), background mail, and unknown emails lock exactly like real ones. → `auth_service.dart:920`

How do you stop brute-forcing email codes? 8 digits, 5 guesses per code, at most 10 codes per account per day, all counted in the database. That's about 0.02 % a year against one account. → `auth_service.dart:1145`, `:1200`

How do you stop replayed messages between phones? Each sender has a counter and a 64-message window, plus a ±2 minute clock check. → `replay_guard.dart:25`, `secure_messenger.dart:131`

What about a fake member uploading records? They can't produce the session tag without the invite code, so the console flags them as "not proven a member". Votes count people, not uploads. → `session_credentials.dart:43`, `admin_service.dart:694`

How do you know the audit log wasn't edited? Triggers refuse edits; each row's HMAC covers the previous row; the key isn't in the database; and the latest entry is anchored with every backup. → `audit.dart:125`

Is the password check sending my password to a third party? No. Only the first 5 characters of a SHA-1 hash leave the server; HIBP returns hundreds of matches and the comparison happens on our side. → `breach_check.dart:45`

What happens offline? Phones keep working together. Uploads wait in the outbox, already sealed, and go when there's signal. You stay signed in. → `outbox.dart:180`

What if the phone's clock is wrong? The app learns the server's time from the `Date` header and signs with that. → `api_client.dart:84`, `account_provider.dart:94`

Why Dart on the server? One language end to end, and the sealing and device-proof code is literally the same package on both sides (`packages/echopin_protocol`), so they can't drift apart.

What's not done? Bluetooth between real phones still needs field testing; the custodian key must be created before real use; verified-account badges inside sessions are not built.

Numbers to remember

Access token	30 min
Refresh token	30 days (7 without a keystore), rotated every use
Device-proof window	±5 min

Phone message clock window	±2 min
Replay window	64 messages per sender
Flood limit	5 messages, 1 more every 500 ms
Session size	20 members
Email code	8 digits, 15 min, 5 guesses, 10 codes/day
Lockout	after 5 failures, 15 min doubling to 24 h
Argon2id	19 MiB, 2 passes
Break-glass reason	≥ 10 characters, 30 per hour
Public endpoint floor	600 ms
Retention	records 365 d, diagnostics 90 d, closed reports 730 d
Backups	daily, keep 14, AES-256-GCM in 1 MiB frames
Tests	213 app · 62 server · 6 protocol · 30 live checks